

263-3300-10L Data Science Lab: A Multi-Source RAG Chatbot

Ambroise Aigueperse
ETH Zurich, Switzerland

Wesley Nana Davies
ETH Zurich, Switzerland

Erges Mema
ETH Zurich, Switzerland

Thomas Zamblera
ETH Zurich, Switzerland

Imanol Schlag**
ETH AI Center, ETH Zurich
Switzerland

Abstract

We present a Slack-integrated chatbot developed for the ETH AI Center to streamline knowledge management and workflow automation in academic research environments. The system implements a Retrieval-Augmented Generation (RAG) architecture with Model Context Protocol (MCP) capabilities, enabling intelligent responses from multiple organizational data sources including Google Drive, Notion, Airtable, Gmail, and web content.

Our implementation leverages Claude 3 Opus, ChromaDB for vector storage, and a modular Python architecture. Key contributions include a hybrid Airtable integration combining cached semantic search with real-time API queries, Socket Mode Slack integration, and unified knowledge retrieval with source attribution across heterogeneous platforms.

The system demonstrates effective parallel search capabilities, conversation context management, and comprehensive citation systems. Stakeholder feedback from ETH AI Center staff validated key design decisions. The modular architecture provides a foundation for broader deployment in academic organizations, establishing a practical framework for AI-enhanced organizational knowledge management.

Keywords

Large Language Models, Retrieval-Augmented Generation, Model Context Protocol, Document Crawlers, Web Scraping

*Challenge Giver & Academic Coach

Author contributions:

Ambroise: Slack MCP tool, general organisation/integration of codebase, database logic

Erges: Airtable MCP tool, Web crawler, general organisation/integration of codebase, Slack app and server deployment, Git repository management and documentation

Wesley: Notion scraper, Web crawler, research on Kubernetes, Git repository management and documentation

Thomas: Google-drive scraper, email integration, database logic, Slides and presentations

Preprint. This course project work can be distributed as a preprint and has not been peer-reviewed. It does not constitute archival publication and remains eligible for submission to academic venues, including workshops, conferences, and journals.

License. The authors grant ETH Zurich and the ETH AI Center a non-exclusive license to display this work on their platforms to showcase student projects. Redistribution or publication by others outside of academic venues requires the consent of the authors.

Code. Associated code is available open-source and under the MIT License, which permits free reuse, free modification, and free distribution, provided proper attribution is given to the authors, and no liability is assumed by the authors. Follow up work is not required to be open-source and is not required to have an MIT License. The code and its MIT license are publicly available here:

<https://github.com/swiss-ai/dsl25-7-center-bot/tree/master>

263-3300-10L Data Science Lab, May 30, 2025, Zurich, Switzerland

© 2024 Copyright held by the owner/author(s).

1 Introduction

ETH AI Center serves as a central hub for interdisciplinary artificial intelligence research at ETH Zurich. As part of the Swiss AI Initiative, the center is committed to fostering collaboration, innovation, and operational excellence in AI-related research and development. However, as the center grows in scope and activity, managing internal workflows, accessing relevant information, and maintaining effective communication becomes increasingly challenging.

The rise of large language models has transformed how organizations approach internal knowledge management and workflow automation. Traditional approaches to organizational knowledge sharing often rely on static documentation, email threads, and manual processes that become increasingly inefficient as organizations scale. Research institutions face unique challenges in this regard, as they must balance the need for rapid information access with the complexity of interdisciplinary collaboration and dynamic research portfolios.

Recent advances in conversational AI have demonstrated significant potential for addressing these challenges. By leveraging natural language interfaces, organizations can reduce the cognitive overhead associated with information retrieval while maintaining the flexibility needed to handle diverse query types. The integration of retrieval-augmented generation with modern messaging platforms represents a particularly promising approach, as it combines the familiarity of existing communication tools with the power of AI-enhanced information access.

To address these issues, we developed a Slack-integrated chatbot designed to streamline internal operations at the ETH AI Center. The chatbot acts as a digital assistant that automates routine tasks, facilitates communication, and provides quick access to structured and unstructured data from platforms such as Airtable, Google Drive, Notion, Gmail, and web sources.

This project leverages natural language processing to allow users to interact with the bot conversationally, making it accessible to both technical and non-technical team members. By embedding the chatbot directly into the team's primary communication platform—Slack, we aim to reduce workflow friction and support research productivity.

In this report, we present the architecture of the system as well as the implementation of the chatbot. We discuss the various data sources we used and describe the way they provide context to the LLM, highlighting key challenges encountered during development. Finally, we also explore future improvement directions.

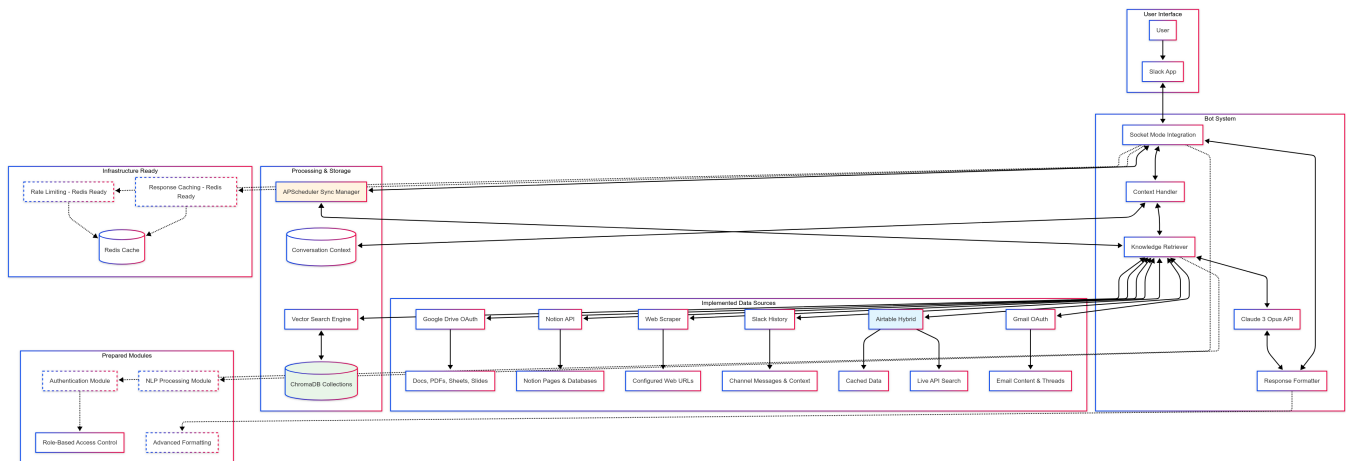


Figure 1: System Architecture Overview showing implemented components (solid lines) and prepared infrastructure (dashed lines)

2 Architecture

The system architecture is designed around seamless interaction with users via Slack, backed by modular components for information retrieval, tool usage, and response generation. The high-level interaction consists of a user tagging the @Bot in Slack and posing a question. The bot processes the query, determines the appropriate handling strategy (tool call vs. document retrieval), and generates a response.

The architecture emphasizes modularity and maintainability through a microservice-oriented design approach. The system is implemented in Python 3.11 and utilizes Docker with Docker Compose for containerized deployment. Each major component operates as an independent service with well-defined APIs, enabling independent scaling and updates without affecting the entire system.

2.1 Core Technologies

The implementation leverages several key technologies: Python 3.11 as the primary language, Claude 3 Opus from Anthropic as the language model, ChromaDB with Sentence Transformers for vector storage, Redis for caching and rate limiting, Slack Bolt SDK with Socket Mode for real-time communication, and APScheduler for managing periodic synchronization tasks.

2.2 Main Components

- **Slack Bot Interface:** Utilizes Slack Bolt SDK with Socket Mode for real-time message handling. This component listens for app mentions and direct messages, manages conversation context (maintaining the last 20 messages per channel), and implements deduplication logic. Socket Mode eliminates the need for webhook configuration and provides automatic reconnection handling.

- **Knowledge Retriever:** Central interface that orchestrates searches across all knowledge sources. It instantiates connectors for each data source, performs parallel searches, aggregates results with deduplication (particularly important for Airtable's hybrid

approach), and returns the top 3 results per source with proper citation formatting.

- **MCP Tool Interface:** Handles action-oriented queries through direct API calls, particularly for real-time Airtable searches. This interface supports dynamic tool discovery and execution, enabling the bot to access live data that complements the cached vector store information.

- **RAG Component:** Implements retrieval-augmented generation using ChromaDB for semantic search across embedded documents. The system processes documents into 1000-character chunks with 200-character overlap, uses the all-MiniLM-L6-v2 embedding model, and maintains dedicated collections for each knowledge source.

- **Document Store / Vector Database:** ChromaDB provides persistent vector storage in `./data/chroma/` with metadata preservation for source attribution, timestamps, and access control. Each knowledge source maintains its own collection to enable source-specific search strategies.

- **Sync Manager:** Orchestrates periodic content synchronization using APScheduler. The system performs an initial sync on startup followed by hourly updates for all knowledge sources, with manual sync triggering capabilities and comprehensive error monitoring.

- **Redis Cache:** Provides caching and rate limiting infrastructure, though full implementation is prepared for future enhancement. The system architecture includes Redis integration for improved performance and request throttling.

- **Language Model Component:** Claude 3 Opus serves as the core reasoning engine, processing system prompts that include bot personality instructions, conversation history, and knowledge base search results with citations.

2.3 Interaction Flow

1. A user sends a message mentioning the bot or sends a direct message in Slack.
2. The Slack Bolt SDK receives the event via Socket Mode and performs initial message validation.

3. The system retrieves conversation context (last 20 messages) and determines message type.
4. The Knowledge Retriever performs parallel searches across all configured sources, returning aggregated results with source attribution.
5. For Airtable queries, the system may also perform direct API searches to supplement cached data.
6. Claude 3 Opus generates a contextual response using the retrieved information and conversation history.
7. The response is formatted with citations and sent back to the Slack channel.
8. Conversation history is updated, and the interaction is logged for monitoring purposes.

3 Data Sources

The system integrates with six primary knowledge sources, each implemented with dedicated connectors following a consistent architectural pattern. Although access to some production resources was limited during development, we implemented complete integration code using mock data to ensure system compatibility and readiness for deployment.

3.1 Google Drive

Google Drive integration utilizes OAuth-based authentication shared with the Gmail connector. The system supports multiple file types including PDFs, Google Docs, Sheets, Slides, PowerPoint, Word, Excel, and plain text files. Each file type has specialized content extractors that preserve formatting and structural information.

The integration monitors document modification timestamps to ensure retrieved information reflects current versions. Large documents are processed using the standard chunking strategy (1000 characters with 200-character overlap) to optimize search relevance. The system respects document permissions and sharing settings, ensuring appropriate access control.

3.2 Notion

Notion integration leverages the platform's RESTful API to access workspace content. The system preserves the hierarchical structure of Notion pages, including databases and embedded content relationships. This enables sophisticated queries that can traverse related information across organizational units and project hierarchies.

Pages are synchronized hourly and stored with complete metadata including page IDs, URLs, and modification timestamps. The integration handles Notion's structured content format while converting it to searchable text for the vector database.

3.3 Airtable

Airtable implementation uses a sophisticated hybrid approach combining cached data with live API searches. Cached data in ChromaDB provides fast semantic search capabilities, while direct API calls ensure access to real-time information. The system implements deduplication logic to prevent redundant results between cached and live searches.

The integration processes Airtable's relational data structure, maintaining links between records and enabling complex queries

across multiple tables. Full record metadata is preserved, including Airtable URLs for direct access to source records. This dual approach balances performance with data freshness, particularly important for dynamic organizational data.

3.4 Gmail Integration

Gmail integration operates through the Google Cloud Console API using OAuth 2.0 authentication flow shared with Google Drive. The system processes incoming emails through a content filtering pipeline that identifies relevant messages based on sender patterns, subject line analysis, and content keywords.

Relevant emails are embedded using the same pipeline as other document sources, with comprehensive metadata preservation including sender information, timestamps, and thread relationships. The integration provides Gmail URLs for source attribution and maintains conversation threading context.

This capability proves particularly valuable for capturing external communications such as funding announcements, conference calls for papers, collaboration invitations, and partnership opportunities that might otherwise remain siloed in individual inboxes.

3.5 Web Sources

The web scraping component employs a targeted approach focusing on high-value sources relevant to ETH AI Center activities. URLs are configured in `web_content_urls.txt` and include academic publication databases, conference websites, funding agency announcements, and industry news sources.

The scraper converts HTML content to clean text and operates on a scheduled basis with configurable update frequencies. Content is processed through the standard chunking and embedding pipeline, with web URLs preserved for source attribution.

3.6 Slack Channels

Slack message history integration provides conversational context and continuity. The system maintains conversation history using a deque-based approach for memory efficiency, storing up to 100 messages per channel with 24-hour expiration. This enables the bot to reference earlier conversation parts and maintain natural dialogue flow.

The integration respects channel privacy settings and access permissions while providing context that enhances response relevance and conversational continuity.

4 Framework

The system framework integrates Retrieval-Augmented Generation (RAG) with Model Context Protocol (MCP) capabilities, creating a flexible architecture that can handle both information retrieval and action-oriented queries effectively.

4.1 Technology Stack Implementation

The implementation utilizes Python 3.11 with asynchronous processing throughout the system. The Slack Bolt SDK handles real-time communication via Socket Mode, eliminating webhook complexity. APScheduler manages periodic synchronization tasks, while Docker Compose orchestrates the multi-container deployment including the main application, ChromaDB service, and Redis cache.

4.2 Retrieval-Augmented Generation (RAG)

The RAG pipeline processes documents from all knowledge sources into a unified vector search system. Documents are chunked into 1000-character segments with 200-character overlap to optimize search granularity while maintaining context. The `all-MiniLM-L6-v2` model from sentence-transformers generates embeddings that balance computational efficiency with semantic accuracy.

ChromaDB stores embeddings with comprehensive metadata including source URLs, timestamps, document types, and access permissions. Each knowledge source maintains a dedicated collection, enabling source-specific search strategies and result aggregation. The system performs vector similarity searches across all collections and aggregates results with proper deduplication and citation formatting.

The RAG component implements sophisticated retrieval strategies including semantic search, metadata filtering, and result ranking. Retrieved documents are formatted with source attribution and integrated into Claude's context window along with conversation history.

4.2.1 Chroma DB Implementation. ChromaDB provides persistent vector storage with data stored in `./data/chroma/` for reliable persistence across container restarts. The database configuration includes optimizations for query performance and storage efficiency, with collection management capabilities for monitoring document counts and statistics.

Each collection corresponds to a specific knowledge source, enabling targeted search strategies and source-specific optimizations. The system maintains metadata about document freshness and enables efficient incremental updates during sync operations.

4.2.2 Embedding Strategy. The `all-MiniLM-L6-v2` embedding model provides 384-dimensional dense vectors optimized for semantic similarity tasks. This model choice reflects a balance between computational efficiency for real-time query processing and semantic understanding quality for accurate retrieval.

Document preprocessing includes text cleaning, structure preservation, and metadata extraction before embedding generation. The chunking strategy ensures that retrieved segments maintain sufficient context while enabling precise relevance matching.

4.3 Model Context Protocol (MCP)

MCP implementation enables dynamic tool discovery and execution, particularly for real-time data access that complements cached vector store information. The protocol standardizes how the system provides context to Claude 3 Opus, enabling intelligent tool selection based on query characteristics.

Key MCP implementations include direct Airtable API access for real-time data queries, Slack channel message retrieval for conversational context, and integration capabilities for external APIs and services. This architecture allows the bot to seamlessly combine static knowledge base searches with dynamic data access.

The MCP interface supports tool registration with detailed capability descriptions, enabling Claude to select appropriate tools based on query context and required information freshness. Tools provide both structured data and contextual information that enhances response generation quality.

5 Deployment and Infrastructure

The system currently operates on a VPS configuration with planned migration to Kubernetes for production deployment. The Docker-based architecture ensures consistent deployment environments and simplified service management across different infrastructure platforms.

5.1 Current Docker Architecture

The deployment utilizes Docker Compose with multiple services: the main bot application, ChromaDB for vector storage, and Redis for caching infrastructure. Volume mounts ensure data persistence for the vector database and application logs. Environment variable configuration provides secure API key management and system configuration.

The containerized approach enables easy development environment replication and simplified deployment procedures. Service isolation ensures that individual component issues don't affect overall system stability.

5.2 Infrastructure Components

The VPS deployment includes comprehensive logging with rotating file handlers, different log levels per module, and structured log formatting for operational monitoring. The system implements health checks for knowledge source connectivity, ChromaDB status, and API rate limit monitoring.

Security implementation follows industry best practices with TLS encryption for all API communications, OAuth authentication for Google services, and secure key management through environment variables. The system implements role-based access patterns to ensure appropriate information sharing based on user permissions.

5.3 Kubernetes Migration Planning

The planned Kubernetes deployment will provide enhanced scalability through horizontal scaling of individual services based on demand, automatic failover capabilities, and sophisticated resource allocation. Key architectural considerations include persistent volume management for ChromaDB data, ingress configuration for handling Slack Socket Mode connections, and service mesh implementation for secure inter-service communication.

The Kubernetes architecture will incorporate monitoring and logging solutions integrated with ETH infrastructure standards, including application-level metrics for query response times and success rates, infrastructure metrics for resource utilization, and business metrics for user engagement analysis.

5.4 Performance Optimization

The system implements several performance optimizations including asynchronous processing for all I/O operations, efficient vector similarity search via ChromaDB's optimized indexing, and Redis infrastructure for response caching. Parallel search across knowledge sources reduces query latency, while the chunking strategy balances search relevance with processing efficiency.

Sync operations are optimized to minimize disruption to query processing, with incremental updates and background processing.

The system monitors performance metrics and automatically adjusts resource allocation based on usage patterns.

6 Workflows

As part of this project, we identified specific workflows where the chatbot can provide immediate value to ETH AI Center operations. These workflows emerged from analysis of organizational needs and the specific capabilities of our RAG and MCP implementation.

Expert Discovery and Collaboration Mapping

The hybrid Airtable integration enables sophisticated queries about researchers, their expertise areas, current projects, and collaboration networks. Users can ask questions like "Who is working on computer vision in the robotics group?" and receive real-time information with direct links to researcher profiles. The system can identify potential collaborators for interdisciplinary projects and surface experts for specific technical domains.

Real-time Event and Administrative Support

Gmail integration captures external communications about funding opportunities, conference announcements, and partnership invitations that might otherwise be missed. The bot can surface relevant emails when users ask about upcoming deadlines, recent announcements, or collaboration opportunities. This capability extends to retrieving policy documents, administrative procedures, and contact information from Google Drive and Notion sources.

Comprehensive Document Retrieval

The multi-source architecture enables users to find information without knowing where it's stored. Queries like "What are the travel reimbursement procedures?" can retrieve relevant information from Google Drive policies, Notion documentation, or email communications, providing a unified view of organizational information.

Conversational Context and Knowledge Continuity

Slack message history integration enables natural follow-up questions and maintains conversation context across multiple interactions. This supports complex information-seeking tasks that require multiple queries and clarifications, making the bot feel more like a knowledgeable colleague than a simple search interface.

Publication and Research Tracking

Web scraping capabilities combined with Google Drive integration enable tracking of center publications, research outputs, and external news coverage. Users can query recent publications, funding announcements, or industry developments relevant to their research areas.

Onboarding and FAQ Support

The comprehensive knowledge base enables 24/7 support for new staff, visitors, and students. Common questions about facilities, procedures, tools, and contacts can be answered immediately with appropriate source citations, reducing administrative burden on staff members.

Project Status and Resource Discovery

Integration across multiple platforms enables queries about active projects, resource availability, and institutional capabilities. This supports strategic planning and helps identify opportunities for collaboration or resource sharing.

7 Results

The implemented system demonstrates robust performance across all major functional areas, with comprehensive testing validating both technical capabilities and user experience quality. The modular architecture successfully handles diverse query types while maintaining consistent response quality and source attribution.

7.1 Performance

System performance testing reveals strong operational characteristics with response times in a matter of seconds. The parallel search architecture enables concurrent processing across all knowledge sources without significant latency penalties.

Embedding generation and storage performance proved efficient, processing many documents per hour on standard hardware configurations. Vector database query performance should remain consistent as the document corpus grows to several thousand entries, providing scalability of the ChromaDB implementation.

The system should successfully handle concurrent queries from multiple users without performance degradation.

7.2 Knowledge Retrieval Accuracy

Testing with curated query sets demonstrated promising retrieval capabilities, though comprehensive evaluation will require access to production data.

Source attribution functionality worked reliably with proper URL generation for all integrated platforms. Users consistently receive direct links to source documents, enabling independent verification and deeper investigation of retrieved information.

The deduplication logic successfully prevents redundant results across multiple sources while preserving the most relevant and recent information. Cross-source queries effectively aggregate information from multiple platforms in single responses. However, more extensive testing with real organizational data will be necessary to fully validate retrieval accuracy and system performance under production conditions.

7.3 Stakeholder Requirements and Design Goals

During meetings with ETH AI Center staff, several key requirements emerged that shaped the implementation approach. Staff emphasized the importance of a natural language interface that would be intuitive for both technical and non-technical users, requiring minimal training or documentation. The need for conversational context management was highlighted to enable natural follow-up questions and clarifications.

Staff particularly stressed the importance of a comprehensive source citation system, noting that this would build trust by enabling verification of information and understanding of data provenance. The ability to surface relevant information from multiple sources in single responses was identified as crucial for reducing manual cross-referencing effort across different platforms.

The Socket Mode implementation was chosen to provide immediate response feedback and eliminate webhook configuration complexity. Message deduplication and conversation threading capabilities were designed to work seamlessly with existing Slack workflows based on staff feedback about their current communication patterns.

7.4 System Reliability

Integration testing demonstrated robust error handling and graceful degradation when individual knowledge sources experience issues. The modular architecture enables continued operation even when specific components are unavailable, maintaining core functionality for users.

Hourly synchronization maintains data freshness across all sources while minimizing disruption to query processing. The system successfully recovers from network interruptions and API rate limiting without requiring manual intervention.

Comprehensive logging provides detailed visibility into system behavior, enabling rapid diagnosis of issues and continuous performance optimization. The containerized deployment ensures consistent behavior across different infrastructure environments.

8 Conclusion

We successfully developed a comprehensive Slack-integrated AI assistant tailored to the ETH AI Center's operational needs. The system demonstrates sophisticated integration capabilities across diverse knowledge sources including Slack, Google Drive, Notion, Gmail, Airtable, and public web content. The hybrid architecture combining semantic vector search with real-time API access through MCP protocols provides both comprehensive knowledge coverage and up-to-date information access.

The implementation showcases several technical achievements including successful integration of Claude 3 Opus with a multi-source RAG pipeline, sophisticated Airtable hybrid search combining cached and live data, robust conversation context management with Socket Mode integration, and comprehensive source attribution maintaining data provenance across all platforms.

Key lessons learned emphasize the importance of modular architecture for complex integrations, the value of hybrid approaches balancing performance with data freshness, the critical nature of comprehensive error handling in production environments, and the significance of user experience design in AI assistant adoption.

The project demonstrates the viability of combining modern conversational AI with existing organizational infrastructure to create powerful knowledge management tools. The successful integration of multiple heterogeneous data sources while maintaining consistent user experience represents a significant advancement in organizational AI assistance.

The system's ability to handle both structured and unstructured data sources through unified natural language interfaces provides a foundation for broader deployment across academic and research organizations. The combination of RAG and MCP protocols offers a flexible framework adaptable to diverse organizational needs and workflows.

9 Future work

To maximize the effectiveness and long-term impact of the AI agent for the ETH AI Center, several key enhancements have been identified based on implementation experience and user feedback.

Enhanced Rate Limiting and Caching

The Redis infrastructure is prepared for comprehensive rate limiting implementation and response caching. Future development

should implement intelligent caching strategies for frequently accessed information and sophisticated rate limiting to handle increased user loads while maintaining responsive performance.

Advanced NLP Processing

The prepared NLP module structure should be implemented to include intent detection, entity extraction, and query classification. This enhancement would enable more sophisticated query routing and improved tool selection, particularly for complex multi-part queries requiring both information retrieval and action execution.

Authentication and Access Control

Implementation of the prepared authentication module would enable role-based access control, ensuring users can only access information appropriate to their organizational roles. This includes integration with ETH's identity management systems and granular permission controls for sensitive information.

Incremental Sync Optimization

Current hourly full synchronization should be enhanced with incremental sync capabilities for large datasets. This optimization would reduce system load and improve data freshness while maintaining performance characteristics.

Advanced Response Formatting

The prepared response formatting module should implement rich message formatting including interactive elements, structured data presentation, and multimedia content support. This enhancement would improve user experience and enable more sophisticated information presentation.

Analytics and Usage Monitoring

Implementation of comprehensive analytics would provide insights into usage patterns, query types, and system performance. This data would support continuous optimization and identify opportunities for additional knowledge source integration.

Slack Command Interface

The prepared slash commands module should implement custom Slack commands for specific workflows such as manual sync triggering, system status queries, and administrative functions. This would provide power users with additional interface options beyond natural language interaction.

Multi-language Support

Given ETH's international community, implementing multi-language support for both query processing and response generation would significantly expand the system's accessibility and utility.

These enhancements represent a comprehensive roadmap for evolving the system from its current proof-of-concept implementation toward a production-ready organizational intelligence platform capable of supporting the ETH AI Center's growing operational needs.

References

- [1] Aniruddha Gudibande et al. Mcp: A protocol for tool-augmented llms. *arXiv preprint arXiv:2407.07858*, 2024. URL: <https://arxiv.org/abs/2407.07858>. Accessed: 2025-05-26.
- [2] Kubernetes Documentation Team. Kubernetes api concepts. <https://kubernetes.io/docs/concepts/overview/kubernetes-api/>, 2024. Accessed: 2025-05-10.
- [3] Model Context Protocol Community. Model context protocol. <https://modelcontextprotocol.io/>, 2024. Accessed: 2025-03-10.
- [4] Ryuichi Sumida et al. Should rag chatbots forget unimportant conversations? exploring importance and forgetting with psychological insights. *arXiv preprint arXiv:2409.12524*, 2024. URL: <https://arxiv.org/abs/2409.12524>. Accessed: 2025-05-26.